



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Resenha do artigo “*Design by Contract*”

Aluno: Bernardo Carvalho Denucci Mercado

Data: 12/05/2026

Professor: João Paulo Aramuni

Matéria: Projeto de Software

Design by Contract

O conceito de *Design by Contract*, busca aumentar a confiabilidade do software por meio de uma ideia simples: tratar o desenvolvimento como um conjunto de contratos bem definidos entre diferentes partes do sistema. Em vez de apenas escrever código funcional, o programador estabelece responsabilidades claras para quem usa um componente (cliente) e para quem o implementa (fornecedor).

Nesse modelo, cada operação do sistema pode ser vista como um acordo. O cliente deve cumprir certas condições para usar uma funcionalidade, enquanto o fornecedor garante um resultado específico caso essas condições sejam respeitadas. Isso ajuda a evitar ambiguidades e melhora a clareza do sistema, já que cada parte sabe exatamente o que esperar da outra.

Para formalizar esses contratos no software, utilizam-se *assertions* (afirmações), que descrevem regras importantes. Existem três principais tipos:

- **Pré-condições:** o que deve ser verdadeiro antes da execução de uma rotina;
- **Pós-condições:** o que deve ser garantido após sua execução;
- **Invariantes de classe:** condições que devem ser sempre verdadeiras para os objetos de uma classe.

Esses elementos permitem especificar com precisão o comportamento esperado do sistema, funcionando tanto como documentação quanto como ferramenta de verificação. Por exemplo, uma função de inserção em uma estrutura de dados pode exigir que ela não esteja cheia (pré-condição) e garantir que o elemento foi inserido corretamente (pós-condição).

Um ponto importante dessa abordagem é a divisão clara de responsabilidades. Se uma pré-condição não é atendida, o erro é do cliente; se a pós-condição falha, o erro é do fornecedor. Isso evita redundância de verificações e substitui o estilo tradicional de “programação defensiva”, que tende a deixar o código mais complexo e difícil de manter.

Além disso, o uso de contratos melhora a forma como lidamos com erros e exceções. Em vez de tratar qualquer problema de forma genérica, o sistema passa a identificar falhas como violações de contratos. Uma exceção, nesse contexto, representa a incapacidade de cumprir um acordo estabelecido.

Outro benefício é na documentação: como os contratos fazem parte do código, eles descrevem o comportamento das rotinas de forma clara e objetiva, sem depender de explicações externas. Isso facilita o reuso de componentes e melhora a comunicação entre desenvolvedores.

Por fim, o *Design by Contract* também contribui para a qualidade do software ao permitir detectar erros mais cedo, durante testes ou execução. As verificações de *assertions* funcionam como uma forma de validar hipóteses feitas pelo programador, tornando o processo de desenvolvimento mais seguro e confiável.

componentes reais. Isso facilita o desenvolvimento orientado a testes e aumenta a confiabilidade do sistema.

O formato hexagonal utilizado para representar essa arquitetura não possui um significado técnico específico, servindo apenas como uma metáfora visual para indicar que a aplicação pode se conectar a múltiplos elementos externos de forma simétrica. Diferentemente das representações em camadas, o hexágono evita sugerir uma hierarquia linear, reforçando a ideia de múltiplas portas independentes.

Ao adotar essa estrutura, o sistema passa a ser organizado em torno de seu núcleo de regras de negócio, enquanto todos os detalhes técnicos são relegados à periferia. Isso resulta em uma aplicação mais flexível, capaz de evoluir com menor impacto diante de mudanças tecnológicas, e mais robusta em termos de manutenção e testes.